

Design Documentation

Laboratory Activity #3

Name (Last, First)	ID Number
Martinez, Gabrielle P.	12413267
Ortha, Gian Lorenzo C.	12414697
Tenorio, Jeroen Ralph I.	12415111
Xu, Kai Wen H.	12415375

1.0 Overview

This project demonstrates a simple client-server communication system using Python sockets. The client collects a user-provided integer and sends it along with its name to the server. The server validates the input, generates its own integer, and sends a reply containing its name and number. Both sides then display their values and the computed sum. To showcase different designs, the project includes a single-client server that handles one connection at a time and a concurrent server that uses threading to support multiple clients simultaneously.

2.0 Design Approach

2.1 Client (client.py)

The client program serves as the entry point for user interaction. It begins by prompting the user to enter an integer between 1 and 100, with the option of entering 999 as a special shutdown signal for the server. Once the input is validated, the client packages the integer along with the client's name in a dictionary and serializes the message using Python's pickle library. The serialized data is then sent to the server over a TCP socket connection. After sending the request, the client waits for the server's response. When the reply arrives, it contains the server's name and a randomly generated integer. The client displays its own name, the server's name, both numbers, and their computed sum. Finally, the connection is closed, ensuring that system resources are released properly.

2.2 Server (server.py)

The basic server implementation listens persistently on port 4025 and accepts client connections one at a time. When a client message is received, the server first validates the integer. If the value is outside the accepted range, the server prints an error message and terminates gracefully. Otherwise, the server generates its own random integer between 1 and 100. Both the client's and server's details are displayed, including the two integers and their sum. The server then sends a response back to the client containing its name and generated integer before closing the connection. This design demonstrates the fundamental mechanics of handling requests sequentially, one client at a time.

2.3 Concurrent Server (server_concurrent.py)

The concurrent server builds on the structure of the basic server but extends its capability to handle multiple clients simultaneously. Instead of processing one connection at a time, it uses Python's threading mechanism (`_thread.start_new_thread`) to spawn a new thread for each incoming client request. Each thread independently manages communication with its client, ensuring that the server remains available for additional connections while other interactions are still ongoing. This design highlights the scalability of socket programming and illustrates how concurrency can be used to make the server more responsive in multi-client environments.

3.0 Communication Flow

The interaction between the client and server follows a straightforward sequence. The client begins by prompting the user for input and then sends a serialized message containing its name and chosen number to the server. Upon receiving this message, the server validates the input. If the integer is valid, it generates a random number, creates a response message containing its own name and number, and sends this back to the client. If the input is invalid, the server instead prints a termination message and shuts down. The client receives the response, displays the names, numbers, and their sum, and then terminates. Both programs close their sockets at the end of execution, ensuring a clean shutdown of the connection.

4.0 Assumptions & Choices

Several design decisions were made to simplify implementation and testing. The programs communicate over port 4025, which, while below the suggested range of 5000 or higher, was chosen for convenience in local runs. A special client input, 999, was introduced as a manual shutdown trigger for the server, providing a convenient way to stop execution without external commands. Data exchanged between client and server is serialized using Python's pickle library for ease of implementation. Finally, the server's integer values are generated using Python's built-in `randint(1,100)` function, which provides sufficient randomness for the purposes of this assignment.

5.0 Execution Documentation

Execution of the system is demonstrated through several sample runs. In the first scenario, the client enters a valid number within the accepted range. The server generates its own number, both sides display their names and values, and the correct sum is shown. In the second scenario, the client enters a number outside the valid range, triggering the server to print an error message and terminate gracefully. In the third scenario, the concurrent server is used to show how two or more clients can connect and interact simultaneously, each being handled by separate threads without blocking one another. Screenshots of these runs are included to document the system's functionality.

```
C:\Windows\System32\cmd.exe
C:\Users\Kai Wen Xu\Downloads\drive-download-20251006T091114Z-1-001>python client.py
Please enter a number from 1 and 100: 32
Client name: Wheezy Joe Vinaigrette
Server name: Baby Oil Pealike

Client number: 32
Server number: 29
Sum: 61

C:\Users\Kai Wen Xu\Downloads\drive-download-20251006T091114Z-1-001>python client.py
Please enter a number from 1 and 100: 18
Client name: Wheezy Joe Vinaigrette
Server name: Baby Oil Pealike

Client number: 18
Server number: 94
Sum: 112

C:\Users\Kai Wen Xu\Downloads\drive-download-20251006T091114Z-1-001>

C:\Windows\System32\cmd.exe
C:\Users\Kai Wen Xu\Downloads\drive-download-20251006T091114Z-1-001>python client.py
Please enter a number from 1 and 100: 66
Client name: Wheezy Joe Vinaigrette
Server name: Baby Oil Pealike

Client number: 66
Server number: 57
Sum: 123

C:\Users\Kai Wen Xu\Downloads\drive-download-20251006T091114Z-1-001>python client.py
Please enter a number from 1 and 100: 50
Client name: Wheezy Joe Vinaigrette
Server name: Baby Oil Pealike

Client number: 50
Server number: 62
Sum: 112

C:\Windows\System32\cmd.exe - python server.py
Microsoft Windows [版本 10.0.26100.6584]
(c) Microsoft Corporation. 保留所有权利。

C:\Users\Kai Wen Xu\Downloads\drive-download-20251006T091114Z-1-001>python server.py
Server of Baby Oil Pealike listening on port 4025
Client name: Wheezy Joe Vinaigrette
Server name: Baby Oil Pealike

Client number: 32
Server number: 29
Sum: 61

Client name: Wheezy Joe Vinaigrette
Server name: Baby Oil Pealike

Client number: 18
Server number: 94
Sum: 112

C:\Windows\System32\cmd.exe - python server(extra).py
Microsoft Windows [版本 10.0.26100.6584]
(c) Microsoft Corporation. 保留所有权利。

C:\Users\Kai Wen Xu\Downloads\drive-download-20251006T091114Z-1-001>python server(extra).py
Server of Baby Oil Pealike listening on port 4025
Client name: Wheezy Joe Vinaigrette
Server name: Baby Oil Pealike

Client number: 66
Server number: 57
Sum: 123

Client name: Wheezy Joe Vinaigrette
Server name: Baby Oil Pealike

Client number: 50
Server number: 62
Sum: 112
```

6.0 AI Usage Documentation

Generative AI was not used in any portion of this assignment.